Aufgabe 1: Waldbauern

Einige Waldbauern des Landkreises Ravensburg haben sich zu einer gemeinsamen Vertriebs- und Verwaltungsorganisation WaldRV zusammengeschlossen. WaldRV speichert die Daten der Mitglieder und ihrer Wälder in einer Datenbank.
Entwerfen Sie für diese Aufgabe eine Datenbankstruktur. Zeichnen Sie dazu Entity-Sets, Attribute und Relationen, wie sie der unten stehenden Beschreibung entsprechen in ein Entity-Relationship-Diagramm. Alternativ können Sie auch ein UML-Klassendiagramm mit Klassen, Attributen und Assoziationen zeichnen. Achten sie darauf, ...

- dass Sie ein ER-Diagramm (UML-Diagramm) erstellen sollen und nicht ein Diagramm einer relationalen Tabellenstruktur, d.h. M:N-Relationen sind noch nicht in Verbindungstabellen umgewandelt, es gibt mehrwertige und strukturierte Attribute, und eine Strukturierung in Unterentities (Unterklassen) ist möglich.
- dass Sie Anomalien und Redundanzen vermeiden.
- dass Sie die Art der Relationen in der ausführlichen **(min, max)-Schreibweise** kennzeichnen.

Beschreibung der WaldRV-Datenbank

Unten stehende Beispiel-Informationen sollen in der Datenbank enthalten sein und effizient abgefragt werden können. Informationen, die hier nicht genannt sind, brauchen auch nicht ins ER-Diagramm (UML-Klassendiagramm), selbst wenn Sie sie für eine Datenbank für Waldbauern für sinnvoll erachten, z. B. Informationen über Verkäufe von Bäumen.

Insgesamt sind in WaldRV über 100 Waldbauern zusammengeschlossen, zwei Beispielpersonen seien genannt: Einer der Waldbauern ist Johannes Huber. Er wohnt in "Haselhaus 4, 88255 Baienfurt". Telefonisch ist er unter 0751 554535 und per E-Mail unter j.huber@gmx.de zu erreichen.

Ein anderer Waldbauer heißt ebenfalls Johannes Huber und wohnt in "Gasteig 7, 88255 Baindt". Seine Telefonnummer ist 07502 3677899. Von Internet und E-Mail hält er nichts.

Letzterer besitzt zwei Waldstücke. Das erste heißt Sauruck II und hat die Gauß-Krüger-Koordinaten x=3543895, y=5293670 und liegt auf 693 m Meereshöhe. Es umfasst 15 Hektar, das Terrain ist bergig und der Boden ist steinig.

Das zweite mit Namen Langwasser IV ist 20 Hektar groß und liegt 550 m hoch, das Terrain ist eben, der Boden ist lehmig. Auf der Landkarte ist es zu finden bei den Koordinaten (3545000; 5294000).

Ein anderes Waldstück heißt Hinterbichel ist 30 Hektar groß und liegt 580 m hoch, das Terrain ist hügelig, der Boden ist sandig. Auf der Landkarte ist es zu finden bei x=3547000, y=5292000. Das Waldstück Hinterbichel gehört dem Waldbauern Johannes Huber aus Baienfurt.

Ein bestimmtes Waldstück kann **nicht** mehreren Besitzern gemeinsam gehören.

Die Fichte (Picea abies) ist ein schnell wachsender Nadelbaum und liefert Weichholz.

Die Rotbuche (Fagus sylvatica) ist ein langsam wachsender Laubbaum und liefert Hartholz.

Vom 15.5.2005 bis 17.5.2005 wurden im Waldstück Hinterbichel Bäume gepflanzt und zwar 100 Kiefern, 100 Lärchen und 200 Rotbuchen. die Pflanzaktion kostete 3000 Euro. Die Bäume wurden in einem Abstand von 2 m gepflanzt.

Am 11.3.2010 wurden im Waldstück Langwasser IV Bäume gesetzt und zwar 50 Kiefern und 300 Fichten. Die Kosten lagen bei 2000 Euro. Die Bäume wurden in einem Abstand von 2,5 m gepflanzt.

Am 12.6.2010 wurden im Waldstück Sauruck II Bäume gefällt. Die Rodung kostete 2000 Euro. Neben den unten angegebenen Bäumen fiel noch 25 Raummeter Schwachholz an. Gefällt wurden:

- eine 70-jährige Fichte mit 3,7 Festmeter Holz und Zustand 1a, geschätzter Preis 300 Euro, zur Identifikation bekommt sie die Nummer 17120605,
- eine 80-jährige Fichte mit 3,9 Festmeter Holz und Zustand 2b, geschätzter Preis 200 Euro, zur Identifikation bekommt sie die Nummer 18120605,

- eine 280-jährige Eiche mit 4,2 Festmeter Holz und Zustand 1a, geschätzter Preis 3000 Euro, zur Identifikation bekommt sie die Nummer 53120605,
- usw.

Generalisieren Sie Pflanzaktionen und Rodungen zu einem Oberentity-Set (Oberklasse) "Aktionen". Alle weiteren Aspekte der Datenbank wie z. B. Verkäufe von Bäumen, Abrechnung zwischen den verschiedenen Waldbauern, Steuerabrechnung, EU-Fördermittel, Dünge- oder Pflegeaktionen usw. werden für diese Aufgabe **nicht** berücksichtigt.

Aufgabe 2: Datenbank für Radrouten und Wegweiser (Klausur S 2010)

Im Landkreis Ravensburg gibt es ein Netz von ausgeschilderten Radrouten. Diese sollen in einer Datenbank erfasst werden. Erstellen Sie dazu eine **Datenbankstruktur als Entity-Relationship-Diagramm** (mit Entity-Sets, Attributen und Relationen). Achten sie darauf, dass Sie die Art der Relationen in der ausführlichen **(min, max)-Schreibweise** kennzeichnen.

Beschreibung der Datenbank für Radrouten und Wegweiser

Unten stehende Informationen sollen in der Datenbank enthalten sein und effizient abgefragt werden können. Informationen, die hier nicht genannt sind, brauchen auch nicht ins ER-Diagramm, selbst wenn Sie sie für eine Datenbank für Rad-Wegweiser für sinnvoll erachten.

- Im Landkreis Ravensburg gibt es ein Netz von **Radrouten**. Dabei können sich Routen auch kreuzen oder ein kurzes Stück gemeinsam laufen. Unten sehen Sie einen Kartenausschnitt mit einigen Routen, z. B. *Route G* von *Wangen* nach *Weingarten*.

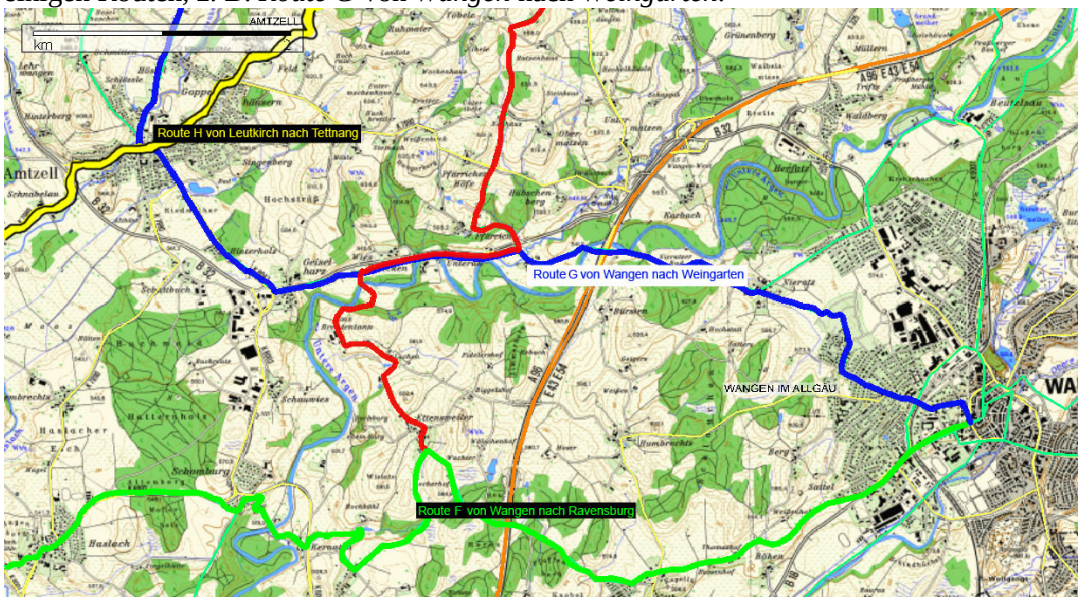

Abbildung 1: Beispiel-Routen

- Die Radrouten sind mit besonderen **Wegweisern** ausgeschildert. Die Wegweiser sind an bestimmten **Standorten** befestigt, vorzugsweise Kreuzungen oder Abzweigungen. Damit die Bauhöfe der Gemeinden wissen, wo sie die Wegweiser aufstellen sollen, wird die Lage der Standorte und die Befestigungsmöglichkeit der Wegweiser daran in der Datenbank beschrieben. Außerdem werden die GPS-Koordinaten des Standorts angegeben.

  Bei jedem Standort will man auch wissen, wie viele **km** er jeweils vom Startpunkt der Routen entfernt ist.
  An den Standorten sind ein oder zwei oder mehrere (wenn sich an dem Standort mehrere Routen kreuzen) Wegweiser angebracht. Zwei Beispiele sehen Sie auf den Fotos unten. Die Radwegweiser sind weiß mit grünem Aufdruck.
  Es gibt zwei Sorten von Wegweisern: Auf der Abbildung 2 sind **Hauptwegweiser** zu sehen. Sie haben zwei Zeilen mit einem Fernziel in der ersten Zeile und einem Nahziel in der zweiten Zeile, dahinter die zugehörigen km-Angaben. Auf der Abbildung 3 sind **Zwischenwegweiser** zu sehen. Sie haben nur einen Pfeil aufgedruckt. Dieser Pfeil kann geradeaus ↑, links ←, rechts →, schräg links ↖, schräg rechts ↗, vorne links ↰ oder vorne rechts ↱ zeigen.

- Bei regelmäßigen Kontrollfahrten werden die Standorte kontrolliert. Dabei werden von einem Standort ein oder mehrere **Fotos** aufgenommen und in der Datenbank gespeichert. Dadurch können die Bauhöfe der zuständigen Gemeinden einfach erkennen, welche Fehler bei den Standorten vorliegen. Manchmal sind auf einem Foto auch zwei nahe beieinander liegende Standorte zu sehen.
- Bei den Kontrollfahrten werden die Fehler oder Bemerkungen zu den Standorten von den Kontrolleuren auch als **Kommentar** in die Datenbank eingetragen.

Abbildung 2: Standort mit Hauptwegweisern **Abbildung 3: Standort mit Zwischenwegweisern**

- Die Gemeinden wollen Fehlerlisten aus der Datenbank generieren können, in der alle Standorte mit fehlenden oder fehlerhaften Wegweisern aufgeführt sind, für die sie zuständig sind.
- Die Information zu einem Standort (hier Standort *F04* an der *Route F Wangen – Ravensburg*) könnte so aussehen (siehe Abbildung 4):

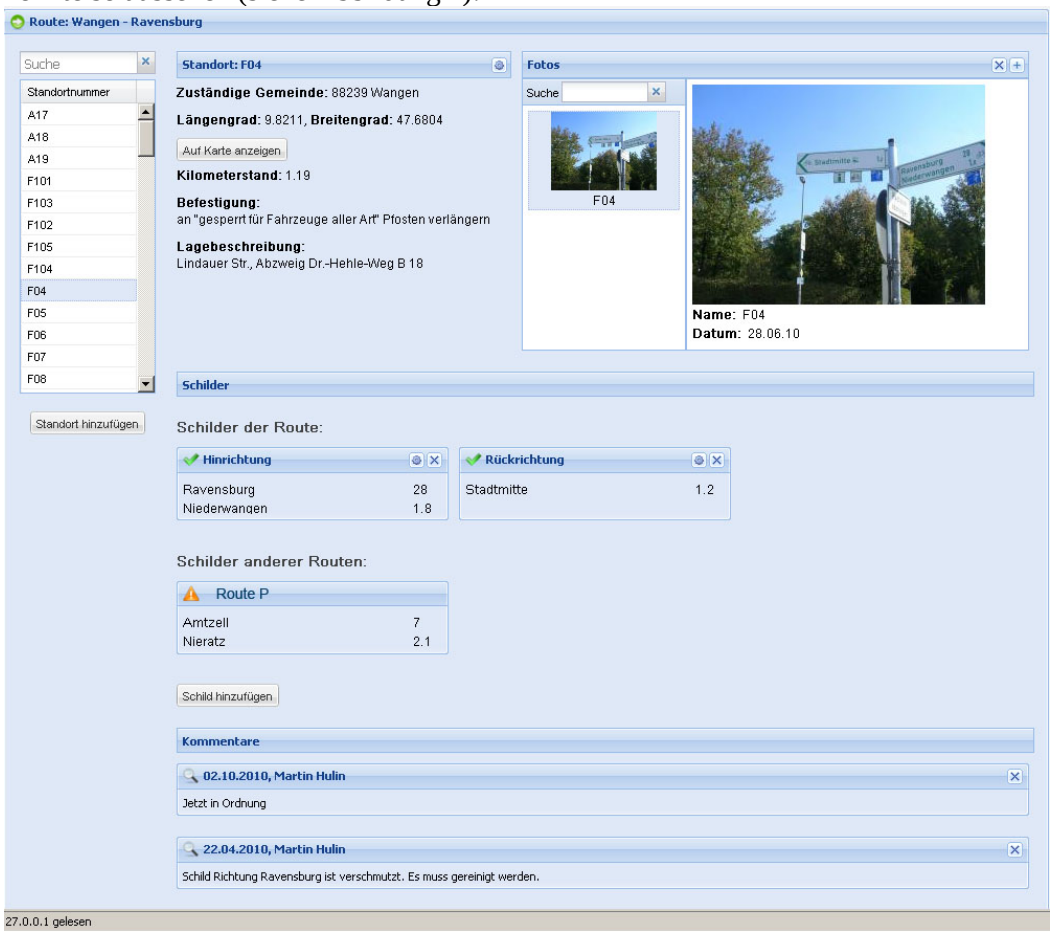

Abbildung 4: Screen-Shot der Web-Anwendung zur Verwaltung von Rad-Wegweisern

Aufgabe 3:

Lesen Sie den Abschnitt über UML-Modellierung aus "Navathe/Elmasri: Grundlagen von Datenbanksystemen" (Skript, Kapitel 2.4). Stellen Sie Gemeinsamkeiten und Unterschiede zu ER-Diagrammen dar.

Aufgabe 4:

Modellieren Sie die Datenbankstruktur für die "Radrouten und Wegweiser" mit UML.

Aufgabe 5:

Modellieren Sie die Datenbankstruktur für die "Radrouten und Wegweiser" mit dem SQL Datamodeler.

Für das Datenenbanksysteme Praktikum ist Aufgabe 1 oder Aufgabe 2 Pflicht!
Abgabetermin siehe MOODLE!

Anleitung für den SQL Datamodeler (siehe auch Video in Moodle)

1. Rechner mit Betriebssystem **Windows** starten (es geht aber auch mit Linux), dazu im K102 im Anmeldefenster "Menüsymbol > Sitzungsart > Windows auf WTS1" ausführen und anmelden.
   Für Linux: Im K102 unter der Oberfläche KDE einloggen
2. "Start > Programs > Development > Oracle > Oracle-CLIENT > SQL-Developer_Data-Modeler"
   Für Linux: KDE-Menu –> Entwicklung > Oracle > SQL Developer (Data modeler)
   (beim ersten Mal: "Java-Version"-Warnmeldung → "Meldung nicht mehr anzeigen" → weiterarbeiten)
3. Menü "Ansicht (View) > Data Modeler > Browser" wählen.
4. In dem gerade geöffneten Browser das Design Untitled_1 mit Klick auf + aufklappen.
5. Rechtsklick auf "Logical" und dann "Show" auswählen.
6. Symbol "New Entitiy" in der Symbolleiste auswählen und auf der Zeichenfläche ein Rechteck aufziehen. In das Dialogformular alle Daten des Entity-Sets wie Name und Attribute eingeben.
7. Für Relationen eines der Relations-Symbole auswählen, erst in das eine, dann in das andere der zu verbindenden Entity-Sets klicken. Mit Doppel-Klick auf die Relation diese anpassen.
8. Mit "File > Data Modeler > Save" sichern Sie Ihr Diagramm als xml-File (bitte auf dem File-Server H:)
   Für Linux: sichern unter /home/stud/loginname
9. Mit der Schaltfläche ">>" (Engineer to Relational Model) wandeln Sie das ER-Diagramm automatisch in ein relationales Tabellen-Schema um.
10. Mit "Ansicht (View) > Data Modeler > DDL File Editor" und dann auf "Generieren" klicken, erzeugen Sie ein SQL Script, mit dem Sie dieses Schema in einer Datenbank implementieren können.

Aufgabenblatt 2: SQL Datendefinition

Bei der ersten Übung, Aufgabe 2 ist ein Entity-Relationship Diagramm für Wegweiser bei Radrouten entstanden. Es könnte z. B. wie folgendes Diagramm aussehen.

Aufgabe:
Wandeln Sie die Entity-Sets und die Relationen (ohne Foto) in die Tabellenstruktur eines relationalen Datenbanksystems um. Benutzen Sie SQL zur Tabellendefinition. Vergleichen Sie Ihre Lösung mit der automatisch generierten Lösung aus Aufgabenblatt 1!

Deklarieren Sie **Primär-** und **Fremdschlüssel** entweder gleich bei der Tabellendefinition oder fügen Sie sie nachträglich in die Tabellen ein.

Tragen Sie in jede Tabelle einen **Beispieldatensatz** ein.

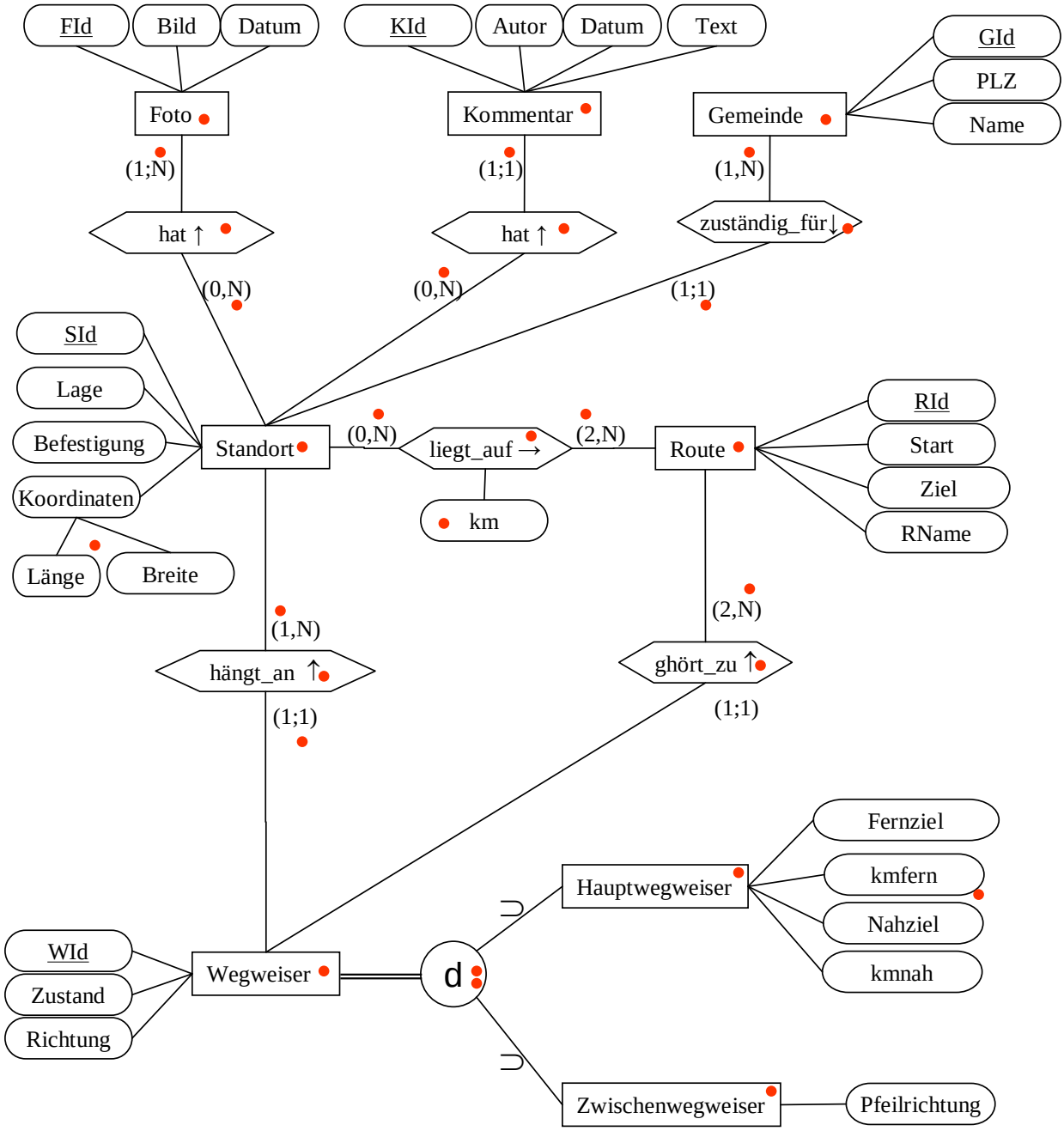

Abbildung 1: Entity-Relationship-Diagramm der Datenbank für Rad-Wegweiser

Zugang zu ORACLE

- Rechner mit Betriebssystem Windows starten (es geht aber auch mit Linux)
- Logon auf PC mit eigener Benutzerkennung
- Aufruf von *SQL-Developer:* `Start > All Programs > Development > Oracle > Oracle-CLIENT > SQL-Developer_Data-Modeler`
- In der linken Seite bei *Connections* auf das grüne + Zeichen klicken.
- Bei *Verbindungsname*, *Benutzername* und *Passwort* den von mir zugewiesenen Namen *dabs_<n>*, n∈{01, 02, ...} eingeben.
- Hostname: `neptun.hs-weingarten.de`
- SID: `namib`
- Auf *Speichern* und dann auf *Anmelden* klicken.
- Ändern des Passwortes: `grant connect to dabs_<n> identified by <neues Passw.>;` Da hier keine sicherheitsrelevanten Daten gespeichert werden, soll ein leicht zu merkendes Paßwort gewählt werden, z. B. Initialen + <n> (z. B. aw12 bei dabs_12 und Anton Weber)

Eingabe von SQL-Anweisungen

- Eine SQL-Anweisung kann direkt in das SQL-Aufgabenblatt eingegeben werden. Mit <F9> wird sie ausgeführt, z. B. `select table_name from user_tables;` (Damit werden übrigens alle erzeugten Tabellen aufgelistet.)
- Mit *Str-D* oder der Radiergummi-Schaltfläche kann das Eingabefenster geleert werden.
- Man kann auch mehrere SQL-Anweisungen nacheinander eingeben und jede mit **;** abschließen. Mit <F5> wird das ganze Skript ausgeführt
- Mit *Str-S* oder der entsprechenden Schaltfläche werden die aktuellen SQL-Anweisungen des SQL-Fensters in eine Datei geschrieben. Mit *Str-O* oder der entsprechenden Schaltfläche kann eine Anweisungs-Datei geladen werden.
- Anweisungsdateien können auch mit jedem anderen Editor erstellt werden.
- Alle Operationen, die Daten ändern, werden erst mit der SQL-Anweisung *commit* endgültig in die Datenbank übernommen. Mit der SQL-Anweisung *rollback* können alle Daten-Aktionen der aktuellen Sitzung rückgängig gemacht werden, aber nicht die Schema-Operationen. Auch für *commit* und *rollback* gibt es Schaltfächen.

Zusätzliche Kommandos, Formatierung

- Mit `desc(ribe)` <*Tabellenname*> erhält man die Spalten einer Tabelle; oder man klickt links auf das + Zeichen vor *Tables* und dann auf die Tabelle.
- Auch die Ausgabe eines SQL-Skripts kann in eine Datei geschrieben werden (Speichern-Schaltfläche).
- Datumsangaben vom Typ *Date* formatieren:

```
select ..., to_char (<Datumsspalte>, 'DD.MM.YYYY'), ... from ...;
insert into ... values
       (..., to_date ('01.01.2222 11:11', 'DD.MM.YYYY HH24:MI'), ...);
```

Pflichtaufgabe für das Datenbanksysteme-Praktikum
Abgabetermin: siehe Moodle!

Aufgabenblatt 3: SQL-DML (mit Relationenalgebra)

1. Schreiben Sie eine Folge von Anweisungen für die **Relationenalgebra** (**nicht** mit **Oracle**, **nicht SQL**): Suche alle Standorte, die an den gleichen Routen liegen, wie der Standort mit der *StID=1*.

2. Verwenden Sie für die folgenden Aufgaben statt des Servers *neptun.hs-weingarten.de* und dem Account *dabs_nn* den Server **fbe-neptun.hs-weingarten.de** und den Datenbank-Account **DABS_nn** bei der Datenbank-Verbindung (Connection) im SQL-Developer. Dadurch bleibt Ihre eigene Datenbankstruktur auf dem alten Server erhalten und Sie können dort bei Bedarf nachschauen.

   Kopieren Sie die Musterstruktur der Wegweiserdatenbank, indem Sie das "SQL-Script für die Wegweiser-Datenbank" *crtWegweiserDB.sql* (zu finden in MOODLE, Kurs Datenbanksysteme) ausführen. Dazu kopieren Sie sich diese Datei in Ihr Verzeichnis, z. B. auf "*H:\dabs*". Dann öffnen Sie die Datei in SQLDeveloper und führen sie aus. Diese Datei ist gleichzeitig die Musterlösung für Aufgabenblatt 2; es gibt aber verschiedene korrekte Lösungen.

   Es werden folgende Tabellen angelegt:
   Route,
   Gemeinde,
   Standort,
   Wegweiser,
   Kommentar,
   liegt_auf (realisiert Relation "liegt_auf" zwischen Standort und Route)

Schreiben Sie interaktive **SQL-Anweisungen** für folgende Aufgaben:

3. Ein zusätzlicher *Standort* wird eingetragen. Er liegt in der Gemeinde *Baienfurt* und auf der Route *Schussental* und zwar 12.1 *km* vom Start entfernt. Er soll am *Lichtmast links* befestigt werden und liegt an der *Hallenbadkreuzung*. Seine GPS-Koordinaten: Länge = *9.655744°*, Breite = *47.828518°*. Tragen Sie die Daten in die Datenbank ein und bestätigen Sie mit *commit*.

4. Jan Jäger stellt fest, dass er bei seiner Kontrollfahrt der Route Schussental seinen Tacho nicht auf 0 gestellt hat, sondern mit 5.5 km gestartet ist. Daher hat er bei allen *liegt_auf* Datensätzen zu dieser Route einen falschen *km-Stand* eingetragen. Korrigieren Sie diese Angaben dauerhaft.

5. Geben Sie den kompletten Inhalt der Tabelle *Gemeinde* aus.

6. Geben Sie Name und Ziel aller Routen aus.

7. Geben Sie Fern- und Nahziel mit Entfernungsangabe von allen Haupt-Wegweisern an, die in Ordnung sind.

8. Suche alle Standorte, die in der Gemeinde *Ravensburg* liegen, und gib deren Lagebeschreibung aus. Verwenden Sie keine Unterabfragen.

9. Gib die Namen aller **verschiedenen** (Hinweis: DISTINCT) Gemeinden aus, in denen ein Standort der Route *Schussental* liegt.

10. Ausgabe aller Zwischen-Wegweiser (Pfeilrichtung, Zustand) der Route *Schussental* mit ihren Standorten (Lagebeschreibung), geordnet nach Richtung (erst Hin-, dann Rück-Richtung) und innerhalb einer Richtung nach km vom Routenstart (aufsteigend).

11. Für jede Route soll die Zahl der Haupt-Wegweiser sowie die durchschnittlichen, die minimalen und die maximalen km-Angaben bei den Fernzielen ermittelt werden.

12. Wiederholen Sie die Aufgabe aber zeigen Sie nur Routen an, die mindestens drei Hauptwegweiser haben. (Hinweis: HAVING)

13. Nutzen Sie reguläre Ausdrücke und den Operator LIKE, um alle Gemeinden im Postleitzahlen-Bereich 882.. zu finden.

14. Unterabfragen: Suchen Sie nach den Namen aller Routen, die den gleichen *Anfang* haben wie die Route *Schussental*.
15. Die Route *Holperroute* wird gelöscht. Löschen Sie auch alle Wegweiser, die zu der Route gehören. Achten Sie auf abhängige Datensätze in Verbindungstabellen. Zum Löschen der abhängigen Datensätze brauchen Sie Unterabfragen. Wie müsste man im Datenbankschema die Fremdschlüssel ändern, damit eine einzige Löschanweisung genügt?
16. Geben Sie alle Haupt-Wegweiser (Fernziel, kmfern) aus, deren Fernziel näher liegt als das am weitesten entfernte Nahziel eines Wegweisers.
17. Suchen Sie alle Standorte, an denen ein falscher Wegweiser hängt, d. h. ein Wegweiser, der zu einer Route gehört, auf der dieser Standort gar nicht liegt.
18. Legen Sie eine Tabelle *Zwischenwegweiser_tab* und einen View *Zwischenweigweiser_view* an, in denen nur Zwischenwegweiser enthalten sind, als Kopie der bestehenden Tabelle *Wegweiser*. Tragen Sie einen neuen Zwischenwegweiser in die Tabelle *Wegweiser* ein. Commit! Prüfen Sie, ob die neuen Daten in *Zwischenwegweiser_tab* und *Zwischenwegweiser_view* übernommen wurden. Löschen Sie die Tabelle *Zwischenwegweiser_tab.*
19. Geben Sie die Zustand und Pfeilrichtung aller Zwischenwegweiser aus. Nutzen Sie dazu den View *Zwischenwegweiser_view*.
20. Gegeben ist die Relation:
    Kommentar a "bezieht sich auf" Kommentar b $\subset$ Kommentar $\times$ Kommentar. Ein Kommentar kann sich nur auf **genau einen** anderen Kommentar beziehen, aber ein Kommentar kann mehrere andere Kommentare haben, die sich auf ihn beziehen.
    Beispiel: Der Kommentar von Manfred vom 1.7.2011 "Wegweiser wurde 50 cm tiefer gesetzt" bezieht sich auf den Kommentar von Robert vom 1.5.2011 "Wegweiser Richtung Friedrichshafen zu hoch angebracht". Ebenfalls auf den Kommentar von Robert bezieht sich der Kommentar von Yvonne vom 5.5.2011 "Wegweiser Richtung Friedrichshafen teilweise durch Baum verdeckt".
    Zeichnen Sie die Relation "bezieht sich auf" in das ER-Modell ein.
    Realisieren Sie die Relation in der Datenbank.
21. Geben Sie Datum und Text aller Kommentare zusammen mit Datum und Text eines evtl. vorhandenen anderen Kommentars aus, auf den sich der Kommentar bezieht. Gibt es keinen Bezug zu einem anderen Kommentar, so soll dieses Feld leer bleiben. (Hinweis: left join)

Hinweis: Die Ausgaben bei den verschiedenen Aufgaben für die Beispieldatenbank finden Sie in Moodle, Kurs Datenbanksysteme in der Datei *A3_Ausgabe.lst.*

Welche der Aufgaben für das Praktikum Datenbanksysteme relevant sind und den Abgabetermin entnehmen Sie bitte der Aufgabe 3
https://www.elearning.hs-weingarten.de/mod/assignment/view.php?id=9859
im Moodle-Kurs Datenbanksysteme.

Normalformen einer Datenbankstruktur

Die Daten eines Busreise-Unternehmens sollen in einer Datenbank gespeichert werden. Die folgende Tabelle zeigt die Daten in einer nicht normalisierten Relation.

Attribut	**Datentyp**	**Beschreibung**
FahrtNr	Zahl	Nummer der Fahrt
BusNr	Zahl	Nummer des Busses
Hersteller	Text	Herstellerfirma des Busses
Plätze	Zahl	Anzahl der Plätze im Bus
Kennzeichen	Text	Polizeiliches Kennzeichen des Busses
Erstzulassung	Datum	Datum der ersten Zulassung des Busses
BusPreis	Zahl	Anschaffungspreis des Busses
FahrtZiel	Text	Ort, an den der Bus fahren soll
FahrtDatum	Datum	Abfahrtsdatum und -Zeit
Fahrzeit	Zahl	Dauer der Fahrt in Minuten
FahrerPersNr	Zahl	Personal-Nr. des Fahrers (pro Fahrt mehrere Fahrer möglich)
FahrerName	Text	Name, Vorname des Fahrers
FahrerAdresse	Text	Adresse eines Fahrers
FahrerGehalt	Zahl	Gehalt eines Fahrers
FahrerTelefon	Liste Text	Telefonnummern des Fahrers
gebuchte Plätze	Zahl	Anzahl der für die Fahrt gebuchten Plätze
FahrtPreis	Zahl	Preis für einen Reisenden
TankKosten	Liste Zahl	Liste Treibstoffkosten für alle Tankstopps der Fahrt

1. Suchen Sie die funktionellen Abhängigkeiten
2. Überführen Sie die Relation in die 1. Normalform. Lagern Sie dazu mehrwertige Attribute in eigene Relationen aus. Achten Sie darauf, das jeweils richtige Attribut als Fremdschlüssel zu verwenden.
3. Überführen Sie das Schema von der 1. Normalform in Boyce-Codd-Normalform.

Dieses Aufgabenblatt ist **nicht** relevant für das Praktikum!

Aufgabe: Zustandsliste der Standorte erstellen

1. Erstellen Sie ein **C-Programm** mit **Embedded SQL**, das für eine **Gemeinde** die **Standorte mit fehlerhaften oder fehlenden Wegweisern** in der folgenden Form ausgibt. Der Name der Gemeinde wird dabei durch den Benutzer eingegeben.

Standorte mit fehlerhaften oder fehlenden Wegweisern in der Gemeinde Baienfurt

Standort Nr. 15 am Punkt (9.649°, 47.815°)
Lagebeschreibung: Am Haldenweg Nähe Hausnr. 10

Fehlende Wegweiser: keine

Fehlerhafte Wegweiser:

Ravensburg	6
Weingarten	1.3
Bad Waldsee	18
Baindt	2.2

Kommentare:

01.10.2011	Wegweiser nach Ravensburg ist verschmutzt.
05.10.2011	Wegweiser nach Bad Waldsee ist verdreht.

Standort Nr.18 am Punkt (9.601°, 47.8224°)
Lagebeschreibung: Kreuzung Baindter Straße mit Brechtstraße

Fehlende Wegweiser:

Zwischenwegweiser rechts

Fehlerhafte Wegweiser:

Zwischenwegweiser links

Kommentare:

01.10.2011 Wegweiser Richtung Ravensburg fehlt, Richtung Baindt verbeult.

...

2. Erstellen Sie ein **Java-Programm** mit **JDBC**, das für eine **Gemeinde** die **Standorte mit fehlerhaften oder fehlenden Wegweisern** in der obigen Form ausgibt. Der Name der Gemeinde wird dabei durch den Benutzer eingegeben.

3. Erstellen Sie ein **Java-Programm** mit **JDBC** und **grafischer Benutzeroberfläche**, das für eine Gemeinde die Standorte mit fehlerhaften oder fehlenden Wegweisern ausgibt. Der Benutzer kann aus der Liste der Gemeindenamen eine Gemeinde auswählen. Daraufhin bekommt er eine Liste aller fehlerhaften Standorte mit StID, Koordinaten und Lagebeschreibung. Wird aus der Standortliste einer ausgewählt, werden die fehlenden und fehlerhaften Wegweiser sowie die Kommentare angezeigt.

Hinweise zu C mit ESQL (unter Windows)**:**

- Erstellen Sie eine Datei mit Endung .pc für Ihr C-Programm mit embedded SQL.
- Für die Verbindung zur Datenbank können Sie sich am Beispielprogramm "Notenausgabe.pc" orientieren. Dieses Programm finden Sie auch in Moodle zum Download im Kurs Datenbanksysteme unter "Materialien".
- Rufen Sie den Pro C Precompiler auf:
  - DOS-Konsole öffnen
  - proc <Filename> eingeben, wobei <Filename> die komplette Pfadangabe enthält, z. B. proc h:\dabs\uebung5\wegweiserESQL\wegweiser_main
  - Ein C-Programm .c wird erzeugt oder Fehlermeldungen werden ausgegeben.
- Starten Sie die Visual C++ Entwicklungsumgebung.
- Erzeugen Sie ein neues Console-Projekt.
- Fügen Sie das vorübersetzte .c-File in das Projekt ein. (Project > Add to Project > Files)
- Fügen Sie das File C:\oracle\client\precomp\lib\orasql10.lib in das Projekt ein.
- Übersetzen und binden Sie nun Ihr Programm.
- Lassen Sie das Programm laufen.
- Hinweis: Oracle Home muss auf "Client Home" eingestellt sein.

Zusätzliche Hinweise zu C mit ESQL unter **Linux:**

- Oracle-Proc im K102 zur Zeit nur unter KDE, Gnome etc. (nicht gentoo) installiert; Pfad: /usr/local/oracle/product/10.2.0/client_1/bin/proc
- erst müssen die notwendigen Variablen gesetzt werden, entweder direkt in der Shell:
  - export PATH=/usr/local/oracle/product/10.2.0/client_1/bin:$PATH
  - export LD_LIBRARY_PATH=/usr/local/oracle/product/10.2.0/client_1/lib
  - oder über den Aufruf: source oracle_ini in einem Shell-Fenster
- dann proc aufrufen mit der nötigen Konfigurationsdatei (hier proc.cfg):
  proc config=proc.cfg parse=partial dateiname.pc (proc.cfg verfügbar in Moodle)
- dann Compilieren/Linken über das nötige Makefile (hier proc.mk):
  make -f proc.mk all TARGET=dateiname

Hinweise zu Java mit JDBC (unter Windows, aber fast genauso unter Linux)**:**

- Verwenden Sie Eclipse als IDE zur Programmentwicklung.
- Erstellen Sie ein neues Java-Projekt
- Fügen Sie mit "Add External JARs" den Oracle JDBC Treiber hinzu. Er befindet sich unter C:\oracle\sqldeveloper\jdbc\lib\ojdbc6.jar
- Die Verbindungsdaten zur Wegweiser-Datenbank unter Oracle sind:
  Connect-String: "jdbc:oracle:thin:@fbe-neptun.hs-weingarten.de:1521:namib"
  Benutzerkennung: "DABS_nn"
  Passwort: <Ihr Passwort> (ursprünglich DABS_nn)
- Beispielprogramme (komplette Eclipse-Projekte) NotenListe.zip, NotenListeGUI.zip finden Sie in Moodle zum Download im Kurs Datenbanksysteme.

Für das **Praktikum** brauchen Sie nur **eine der drei** Aufgaben lösen.
Den **Termin** für die Präsentation Ihrer Lösung entnehmen Sie bitte der Aufgabe in **Moodle**.

1. Sobald ein Wegweiser neu eingetragen wird oder ein Wegweiser an einen anderen Standort gehängt wird, soll mit einem Trigger vorher überprüft werden, ob der Standort des Wegweisers auch auf der Route des Wegweisers liegt.

2. Erstellen Sie einen View *Zwischenwegweiser*, der aus der Wegweiser-Tabelle nur die Zwischenwegweiser ausgibt und natürlich nur die für Zwischenwegweiser relevanten Daten umfasst.
   Erstellen Sie nun einen Instead-Of-Trigger, der es erlaubt, Daten in den View *Zwischenwegweiser* einzufügen. Dabei soll der Wegweiser-Typ automatisch gesetzt werden. Außerdem soll die *Pfeilrichtung* als Voreinstellung auf 'geradeaus' gesetzt werden, wenn sie beim Insert nicht angegeben ist.

3. Wenn ein Wegweiser eingefügt wird und die Angabe *kmfern* leer ist und es bereits einen anderen Wegweiser auf der gleichen Route in die gleiche Richtung mit dem gleichen Fernziel gibt, soll die Angabe *kmfern* automatisch aus der Angabe von *kmfern* des bestehenden Wegweisers und der Angabe *km* in der Tabelle liegt_auf ermittelt werden.

   Beispiel: Auf der *Route Schussental* zeigt der *Wegweiser* am Start als *Fernziel Ravensburg* mit 18 *km* an. Der *km*-Wert in *liegt_auf* ist 0 *km*. Wird jetzt am *Standort* mit der *Id* 4 mit einem *km*-Stand von 6 *km* ein *Wegweiser* mit *Fernziel Ravensburg* neu eingefügt, soll er automatisch 12 *km* als *kmfern* eingetragen bekommen.

Hinweis: Im Oracle SQL Developer können Sie einen Trigger auf zwei Arten eingeben:

- Im SQL Bereich als SQL Script: "CREATE OR REPLACE TRIGGER … END;"
- Mit einem Assistenten: Rechtsklick auf *Triggers* in der linken Spalte und dann *New Trigger* auswählen.

Für das Praktikum präsentieren Sie Ihre Lösungen mindestens zu den Aufgaben 1 und 2 an dem in Moodle genannten Termin.